

Problem 1: Operation Targeting & Risk Optimization

Context & Operational Background

In the midst of an ongoing 40-day conflict, a commanding general has issued an order to perform a cruise missile strike on an enemy military base. However, this base is located near two sensitive non-military sites:

- 1. **A Field Hospital (H)**
- 2. **A Civilian Infrastructure/Facility (C)**

The political and ethical cost of damaging these sites is extremely high. The command headquarters has established a strict safety rule: the probability of a single missile hitting either of the non-military facilities (the union of hitting the hospital or the civilian zone) must be strictly less than 1%.

As an operations research analyst, your objective is to evaluate the risks of direct targeting, perform a 2D parameter optimization to find the safest target coordinates that maximize mission success, and calculate the physical assets (number of missiles) required to secure the operation.

Statistical Modeling of Impact Uncertainty Targeting coordinates are selected as (μ_x, μ_y) . Due to wind, guidance limitations, and other physical variables, the actual landing coordinates (X, Y) are modeled as independent, bivariate normally distributed random variables:

$$X \sim N(\mu_x, \sigma_x^2), \quad Y \sim N(\mu_y, \sigma_y^2)$$

Operational Parameters

- **Missile Precision:** Under current atmospheric conditions:

$$\sigma_x = \sigma_y = 100 \text{ m}$$

- **Safety Constraint:** Let P_{HUS} be the probability of hitting unsheltered/non-military sites $(H \cup C)$:

$$P_{HUS} = P((X, Y) \in H \cup (X, Y) \in C) < 0.01$$

Coordinate Zones The geographic boundaries (in meters) for each facility are defined as follows:

Zone / Site	Code Symbol	X Interval (m)	Y Interval (m)
Main Military Target	M	$[-60, 60]$	$[-60, 60]$
Field Hospital (Risk Zone)	H	$[190, 310]$	$[-60, 60]$

Zone / Site	Code Symbol	X Interval (m)	Y Interval (m)
Civilian Facility (Risk Zone)	C	$[-60, 60]$	$[190, 310]$

Assignment Requirements

Part 1: Risk Assessment of Direct Targeting & Visualization Assuming a direct targeting scenario where the mean aiming point is exactly at the center of the military base ($\mu_x = 0, \mu_y = 0$):

- **Statistical Calculations:**
 - Calculate the probability of hitting the military base (P_M) and the risk of hitting non-military zones (P_{HUS}) analytically using normal Cumulative Distribution Functions (CDFs).
 - Validate these calculations by implementing a Monte Carlo simulation.
- **Constraint Evaluation:** Explicitly declare whether direct targeting violates the 1% collateral risk constraint.
- **Scatter Plot:** Generate a scatter plot showing the coordinates of simulated missile strikes relative to the boundaries of the three rectangular zones.
- **Histogram:** Plot a distribution histogram of the simulated landing locations.

Part 2: Determining the Optimal Targeting Coordinates Because direct targeting may violate safety constraints, you must shift the aiming point (μ_x, μ_y) to protect non-military zones while keeping the military target within range.

Run a full 2D grid search over the range of $\mu_x, \mu_y \in [-500, +500]$ meters with a step resolution of 1 m:

- **Optimization Goal:** Find the optimal aiming coordinate (μ_x^*, μ_y^*) that maximizes the probability of hitting the main target P_M , subject to:

$$P_{HUS} < 0.01$$

- **Feasible Surface Plot:** Generate a 3D Surface Plot showing only the subset of the grid search space (μ_x, μ_y) where the 1% safety threshold is satisfied.

Part 3: Mission Scalability & Repetitive Launch Analysis Assuming the optimal coordinates (μ_x^*, μ_y^*) are selected:

- **Asset Allocation:** If a successful operation requires at least one missile to hit the main military base (M) with a cumulative confidence of $\geq 95\%$, what is the minimum number of missiles (n) that must be fired?
- **Collateral Risk:** Compute the exact analytical probability of hitting at least one non-military structure during an n -missile barrage.
- **Campaign Simulation:** Verify your theoretical findings by running a Monte Carlo simulation of 1000 independent multi-missile strike campaigns. Report the fraction of campaigns that resulted in operational failure (defined as either failing to hit the military target or hitting any non-military structure).

Solution

```
In [1]: # Requirements

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from matplotlib import cm

sigma = 100.0
mu_0 = [0, 0]

Milit = [[-60, 60], [-60, 60]]
Hosp = [[190, 310], [-60, 60]]
Civil = [[-60, 60], [190, 310]]

N = 10000
x = np.random.normal(0, sigma, N)
y = np.random.normal(0, sigma, N)

def get_P(Area : list, mu=[0, 0], sigma=sigma):
    return (norm.cdf(Area[0][1], mu[0], sigma) - norm.cdf(Area[0][0], mu[0], sig

def get_P_Monte_Carlo(Area: list, mu=[0, 0], sigma=sigma, N=N):
    x = np.random.normal(mu[0], sigma, N)
    y = np.random.normal(mu[1], sigma, N)
    # Vectorized condition check (extremely fast):
    inside_x = (x > Area[0][0]) & (x < Area[0][1])
    inside_y = (y > Area[1][0]) & (y < Area[1][1])
    overlap = np.sum(inside_x & inside_y)
    return overlap / N
```

Part 1

```
In [5]: print(f"""
    The probability of the missiles hitting the military base is {100*get_P(Milit, mu_0)}%
    The probability of the missiles hitting any civilian infrastructure is {100*get_P(Civil, mu_0)}%
    """)

fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Subplot 1: Scatter Plot of Missile Attacks -----
axs[0].scatter(x, y, marker="*", s=1, color='blue', alpha=0.5, label=f"Missile i")
axs[0].set_xlim(-500, 500)
axs[0].set_ylim(-500, 500)
axs[0].set_title(rf"Direct Targeting Scatter Plot ($\mu_0=[{mu_0[0]},{mu_0[1]}])")
axs[0].set_xlabel("X Coordinate (m)")
axs[0].set_ylabel("Y Coordinate (m)")
axs[0].legend()
axs[0].grid(True, linestyle=':', alpha=0.6)

# Target Zones
zones = [(Milit, 'red', 'Military (M)'), (Hosp, 'green', 'Hospital (H)'), (Civil, 'blue', 'Civilian (C)')]

for rect, color, label in zones:
    x_min, x_max = rect[0][0], rect[0][1]
    y_min, y_max = rect[1][0], rect[1][1]
```

```

# Trace the 4 corners to form a closed box
box_x = [x_min, x_max, x_max, x_min, x_min]
box_y = [y_min, y_min, y_max, y_max, y_min]

axs[0].plot(box_x, box_y, color=color, linewidth=2.5, label=label)

axs[0].axhline(0, color='gray', linestyle='--', linewidth=1.5)
axs[0].axvline(0, color='gray', linestyle='--', linewidth=1.5)

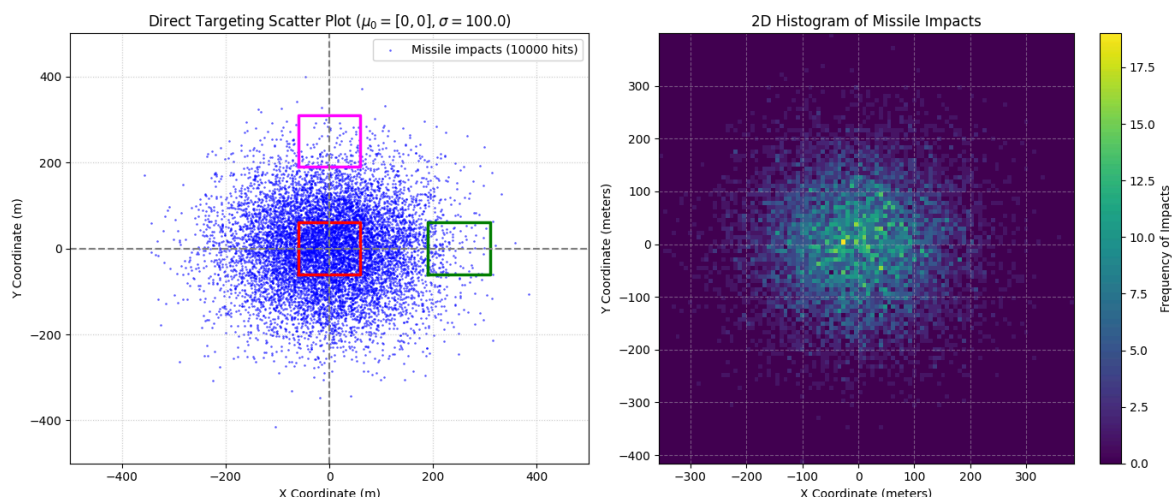
# Subplot 2: 2D Density Histogram -----
counts, xedges, yedges, im = axs[1].hist2d(x, y, bins=100, cmap='viridis')
axs[1].set_title('2D Histogram of Missile Impacts')
axs[1].set_xlabel('X Coordinate (meters)')
axs[1].set_ylabel('Y Coordinate (meters)')
axs[1].grid(True, linestyle='--', alpha=0.5)
fig.colorbar(im, ax=axs[1], label='Frequency of Impacts')

# Tight layout and render
plt.tight_layout()
plt.show()

```

The probability of the missiles hitting the military base is 20.38%

The probability of the missiles hitting any civilian infrastructure is 2.51% which is greater than 1%



Part 2

```

In [6]: # Grid Search Optimization -----
P_optimized = 0
mu_optimized = [0, 0]
grid_points = np.arange(-500, 501, 5)

for X in grid_points:
    for Y in grid_points:
        risk = get_P(Civil, mu=[X, Y]) + get_P(Hosp, mu=[X, Y])
        if risk < 0.01:
            p_m = get_P(Milit, mu=[X, Y])
            if p_m > P_optimized:
                P_optimized = p_m
                mu_optimized = [X, Y]

```

```
print(f"""Optimal Targeting Coordinate: ({mu_optimized[0]}, {mu_optimized[1]})
      Maximized Military Hit Probability: {100*P_optimized:.2f}%
      Collateral Risk at Optimal Point: {100*(get_P(Civil, mu=mu_optimized) + get_
      """)
```

Optimal Targeting Coordinate: (0, 0)
 Maximized Military Hit Probability: 17.99%
 Collateral Risk at Optimal Point: 2.5057%

```
In [ ]: # Feasible Surface plot -----
grid_x = np.arange(-500, 501, 5)
grid_y = np.arange(-500, 501, 5)

X, Y = np.meshgrid(grid_x, grid_y)
Z = np.full(X.shape, np.nan)

for i in range(len(grid_x)):
    for j in range(len(grid_y)):
        mx = X[i, j]
        my = Y[i, j]

        risk = get_P(Civil, mu=[mx, my]) + get_P(Hosp, mu=[mx, my])
        if risk < 0.01:
            Z[i, j] = get_P(Milit, mu=[mx, my])

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

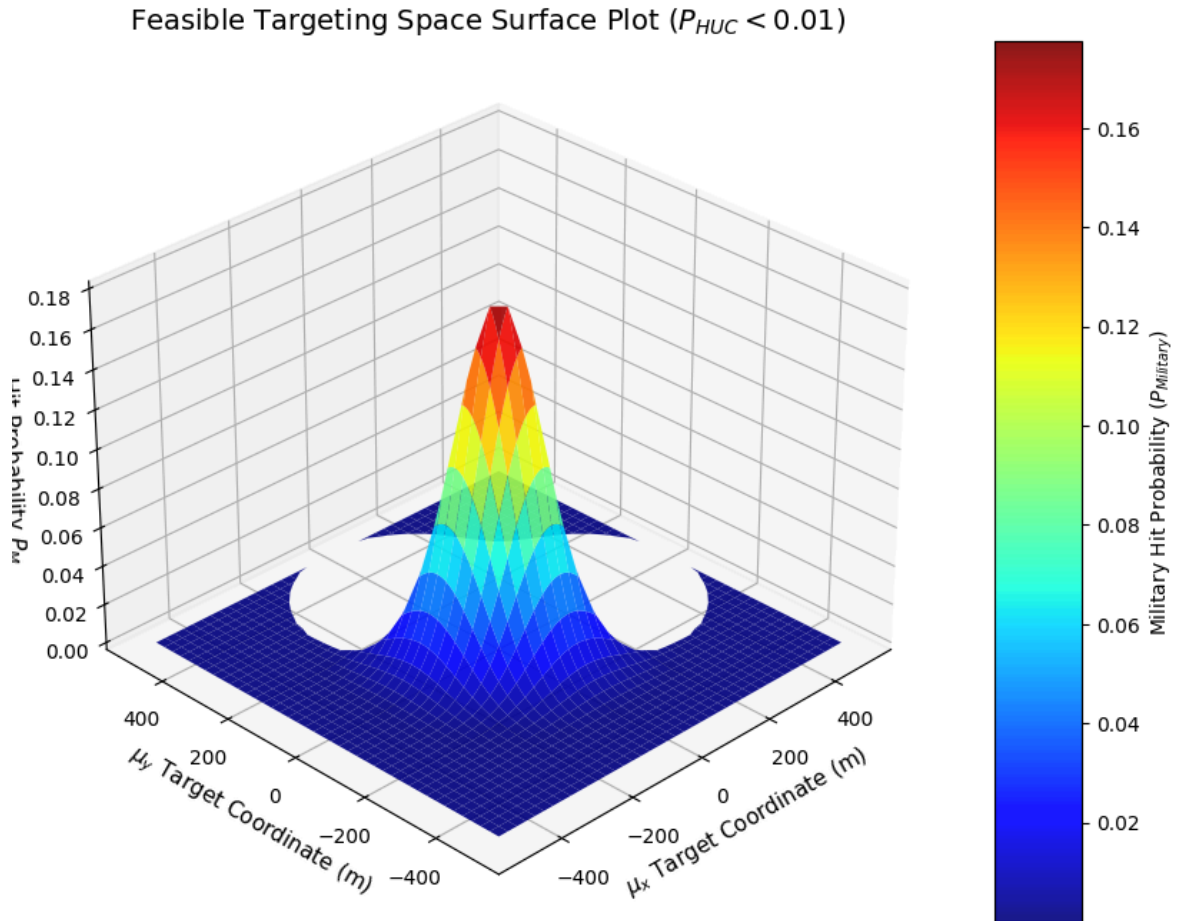
surf = ax.plot_surface(X, Y, Z, cmap=cm.jet, edgecolor='none', alpha=0.9)

fig.colorbar(surf, ax=ax, aspect=10, label='Military Hit Probability ($P_{Milita}

# Labels and Titles
ax.set_title("Feasible Targeting Space Surface Plot ($P_{HUC} < 0.01$)", fontsize=11)
ax.set_xlabel(r"$\mu_x$ Target Coordinate (m)", fontsize=11)
ax.set_ylabel(r"$\mu_y$ Target Coordinate (m)", fontsize=11)
ax.set_zlabel(r"Hit Probability $P_M$", fontsize=11)

# Adjust viewing angle (elevation, azimuth) for optimal visibility
ax.view_init(elev=30, azimuth=-135)

plt.show()
```



Part 3

- We found that the best possible coordinates with a 5 meter error is 17.99% at coordinates (-40, -35). So if we want to have a minimum of 95% success rate, the number of launches should abide the equation $(1 - P_{success})^N < 5\%$. so $N > \frac{\ln 0.05}{\ln(1 - P_{success})}$ which is 16 (larger than 15.1)
- After N hits, the probability of hitting civilian infrastructure will be $1 - (1 - P_{civilian})^N$ which will be equal to 14.177%

```
In [13]: P_success = []
P_pointless = []
for sim in range(1000):
    P_1 = 1 - (1 - get_P_Monte_Carlo(Milit, mu=mu_optimized))**16
    P_2 = (1 - get_P_Monte_Carlo(Hosp, mu=mu_optimized) - get_P_Monte_Carlo(Civi
    P_success.append(P_1)
    P_pointless.append(P_2)
print(f"The Analytical probability of Success after 16 attacks should be {100*(1-(
print(f"The probability of seccess after 16 attacks is {100 * np.mean(P_success)
print(f"The probability of failure with no casaulties is {100 * np.mean(P_pointl
```

The Analytical probability of Success after 16 attacks should be 95.9359%

The probability of seccess after 16 attacks is 97.3874%

The probability of failure with no casaulties is 1.5662%

Problem 2: Fibonacci Sequence and Golden Ratio

Consider the Fibonacci sequence. The general formula for this sequence is:

$$a_1 = 0, \quad a_2 = 1, \quad a_{n+2} = a_{n+1} + a_n$$

Part A Explain at least two applications of the Fibonacci sequence and the Golden Ratio (ϕ) in nature (e.g., in plants, animal structures, or reproduction).

Part B Calculate the first 20 terms of the Fibonacci sequence and store them in a NumPy array.

Part C Calculate the ratios of successive terms (a_{n+1}/a_n) starting from a_3/a_2 and store them in a matrix D .

Part D Perform statistical analysis on matrix D by calculating its mean, standard deviation, median, range, and identifying any potential outliers using the Interquartile Range (IQR) method. Do these statistical measures relate to a well-known mathematical constant?

Part E Plot the elements of matrix D against their indices. Based on your plot, analyze and explain the rate and pattern of convergence of these ratios to their limiting value.

Solution

Part A: Fibonacci Applications in Nature:

- **Plants:** The number of petals on many flowers matches Fibonacci numbers (e.g., lilies have 3, buttercups have 5, daisies have 34 or 55). Seed heads (like sunflowers) and pinecone scales spiral outward in Fibonacci patterns to optimize packing density.
- **Animals:** Shells of creatures like the chambered Nautilus grow in a logarithmic spiral closely approximating the Golden Spiral (built from Fibonacci sequence squares).
- **Reproduction:** The idealized family tree of honeybees follows the Fibonacci sequence (a male bee has 1 parent, 2 grandparents, 3 great-grandparents, 5 great-great-grandparents, etc.).

```
In [15]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# --- Part B: Generate first 20 terms of Fibonacci ---
fib = [0, 1]
```

```

for i in range(2, 20):
    fib.append(fib[-1] + fib[-2])

# --- Part C: Successive Ratios (excluding second over first, i.e., index 1 / in
# We start ratios from index 2 (third term divided by second term) to avoid divi
fib_ratios = []
for i in range(2, 20):
    fib_ratios.append(fib[i] / fib[i-1])

D = np.array(fib_ratios)
print("Part C: Matrix D (Successive Ratios):")
print(D)
print(50 * "-")

# --- Part D: Statistical Metrics for Matrix D ---
mean_val = np.mean(D)
std_val = np.std(D)
median_val = np.median(D)
range_val = np.max(D) - np.min(D)

# Outlier detection using the IQR (Interquartile Range) standard
q1 = np.percentile(D, 25)
q3 = np.percentile(D, 75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

outliers = D[(D < lower_bound) | (D > upper_bound)]
num_outliers = len(outliers)

print("Part D: Statistical Metrics for Matrix D:")
print(f" Mean: {mean_val:.6f}")
print(f" Standard Deviation: {std_val:.6f}")
print(f" Median: {median_val:.6f}")
print(f" Range (Max - Min): {range_val:.6f}")
print(f" Outliers detected: {num_outliers} {outliers.tolist()}")
print(f"\nThe ratios of successive Fibonacci terms converge to the Golden Ratio")
print("-" * 50)

# --- Part E: Plotting the sequence ---
plt.figure(figsize=(10, 6))
# term_indices represents the numerator's position: from term 3 to 20
term_indices = np.arange(3, 21)

plt.plot(term_indices, D, marker='o', color='b', linestyle='-', linewidth=2, lab
plt.axhline((1 + 5**0.5) / 2, color='r', linestyle='--', label=r'Golden Ratio ap

plt.title('Convergence of Successive Fibonacci Ratios to the Golden Ratio', font
plt.xlabel('Term Index ($n$)', fontsize=10)
plt.ylabel('Ratio Value', fontsize=10)
plt.xticks(term_indices)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.show()

```

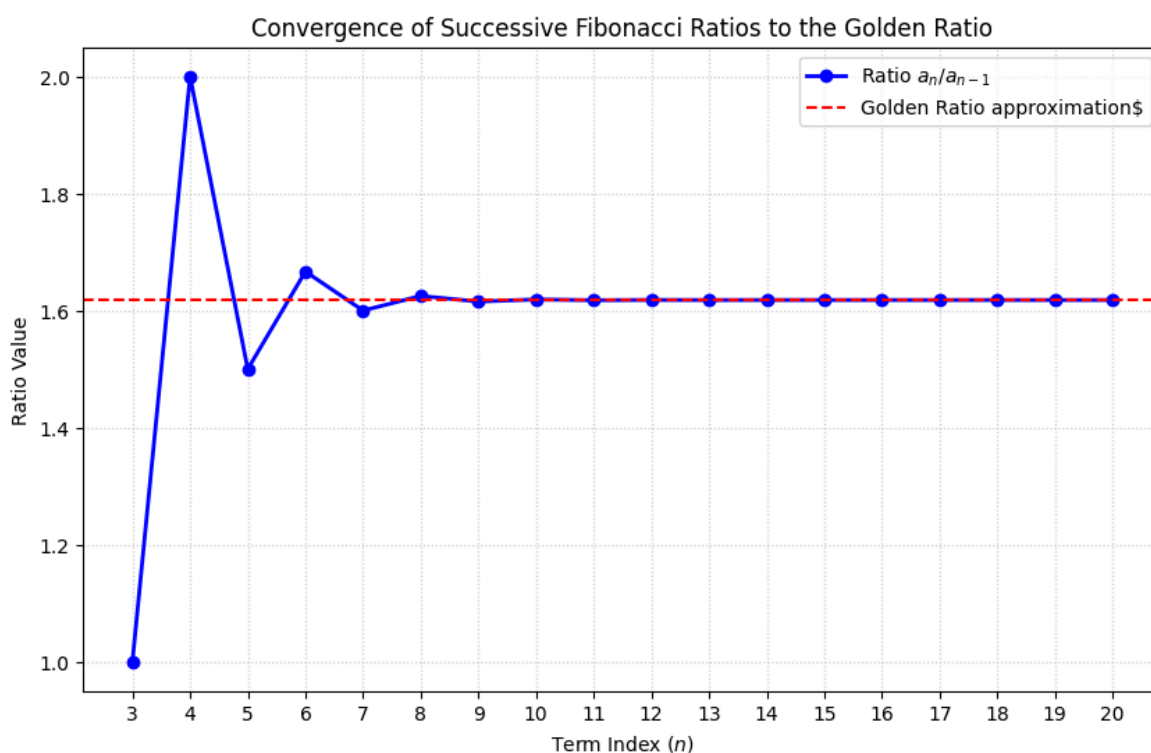
Part C: Matrix D (Successive Ratios):

```
[1.          2.          1.5          1.66666667 1.6          1.625
 1.61538462 1.61904762 1.61764706 1.61818182 1.61797753 1.61805556
 1.61802575 1.61803714 1.61803279 1.61803445 1.61803381 1.61803406]
```

Part D: Statistical Metrics for Matrix D:

```
Mean:          1.600342
Standard Deviation: 0.173030
Median:        1.618034
Range (Max - Min): 1.000000
Outliers detected: 8 [1.0, 2.0, 1.5, 1.6666666666666667, 1.6, 1.625, 1.6153846
153846154, 1.619047619047619]
```

The ratios of successive Fibonacci terms converge to the Golden Ratio (ϕ) = 1.618034



Problem 3: Agricultural Industry

In an agricultural study, two key factors affect the growth of a specific type of plant: the amount of chemical fertilizer X (in kilograms per hectare) and the amount of irrigation water Y (in cubic meters per hectare). The proposed joint probability model for these two variables is given by:

$$f_{X,Y}(x,y) = \begin{cases} K(x+y)e^{-2x-2y}, & x > 0, y > 0 \\ 0, & \text{otherwise} \end{cases}$$

Part A Find the coefficient K such that the function is a valid Joint PDF.

Part B Find the marginal PDFs ($Mpdf_x$ and $Mpdf_y$).

Part C Are the amount of fertilizer (X) and irrigation water (Y) independent variables? Justify your answer with mathematical reasoning.

Part D Calculate the probability that the numerical sum of fertilizer and irrigation water consumed is less than 2 units ($P(X + Y < 2)$).

Part E Calculate the Covariance and Correlation between X and Y .

Solution

```
In [ ]: import sympy as sp

# Setting up the basics
x, y, K = sp.symbols('x y K', real=True, positive=True)
pdf_base = (x + y) * sp.exp(-2*x - 2*y) # Define the non-zero part of the joint

# Part A - Finding the constant K -----
print("--- Part A ---")

double_integral = sp.integrate(pdf_base, (x, 0, sp.oo), (y, 0, sp.oo))
K_val = 1 / double_integral

# Define the fully normalized joint PDF
joint_pdf = K_val * pdf_base

print(f"Value for K is {K_val}")
print(f"Joint PDF f_{{X,Y}}(x,y) = {joint_pdf}")

print("\n")
# Part B: Marginal PDFs -----

marginal_x = sp.integrate(joint_pdf, (y, 0, sp.oo))
marginal_y = sp.integrate(joint_pdf, (x, 0, sp.oo))

print("--- Part B ---")
print(f"Marginal PDF of X, f_X(x) = {sp.simplify(marginal_x)}")
print(f"Marginal PDF of Y, f_Y(y) = {sp.simplify(marginal_y)}")

print("\n")
# Part C: Independence Test -----
print("--- Part C ---")

print("X and Y are independent if f_{{X,Y}}(x,y) = f_X(x) * f_Y(y)")
product_of_marginals = sp.simplify(marginal_x * marginal_y)
are_independent = sp.simplify(joint_pdf - product_of_marginals) == 0

print(f"Product of Marginals f_X(x) * f_Y(y) = {product_of_marginals}")
if are_independent:
    print("Conclusion: X and Y are independent variables.")
else:
    print("Conclusion: X and Y are NOT independent variables.")
```

```

print("\n")
# Part D:  $P(X + Y < 2)$  -----
print("---- Part D ----")

# Integrate over the region:  $x > 0$ ,  $y > 0$ , and  $y < 2 - x$ 
# The limits are:  $x$  from 0 to 2,  $y$  from 0 to  $2 - x$ 
prob_less_than_2 = sp.integrate(joint_pdf, (y, 0, 2 - x), (x, 0, 2))

print(f"Analytical Probability  $P(X + Y < 2) = \{prob\_less\_than\_2\}$ ")
print(f"Numerical value: {prob_less_than_2.evalf():.6f}")

print("\n")
# Part E: Covariance and Correlation -----
print("---- Part E ----")

# Expected values
E_X = sp.integrate(x * joint_pdf, (x, 0, sp.oo), (y, 0, sp.oo))
E_Y = sp.integrate(y * joint_pdf, (x, 0, sp.oo), (y, 0, sp.oo))
E_XY = sp.integrate(x * y * joint_pdf, (x, 0, sp.oo), (y, 0, sp.oo))

cov_XY = E_XY - E_X * E_Y

# Variances for Correlation
E_X2 = sp.integrate(x**2 * joint_pdf, (x, 0, sp.oo), (y, 0, sp.oo))
E_Y2 = sp.integrate(y**2 * joint_pdf, (x, 0, sp.oo), (y, 0, sp.oo))

var_X = E_X2 - E_X**2
var_Y = E_Y2 - E_Y**2

# Correlation coefficient
corr_XY = cov_XY / sp.sqrt(var_X * var_Y)

print(f"E[X] = {E_X}")
print(f"E[Y] = {E_Y}")
print(f"E[XY] = {E_XY}")
print(f"Covariance(X, Y) = {cov_XY} ({cov_XY.evalf():.6f})")
print(f"Variance(X) = {var_X}")
print(f"Variance(Y) = {var_Y}")
print(f"Correlation(X, Y) = {sp.simplify(corr_XY)} ({corr_XY.evalf():.6f})")

```

--- Part A ---

Value for K is 4

Joint PDF $f_{\{X,Y\}}(x,y) = 4*(x + y)*\exp(-2*x - 2*y)$

--- Part B ---

Marginal PDF of X, $f_X(x) = (2*x + 1)*\exp(-2*x)$

Marginal PDF of Y, $f_Y(y) = (2*y + 1)*\exp(-2*y)$

--- Part C ---

X and Y are independent if $f_{\{X,Y\}}(x,y) = f_X(x) * f_Y(y)$

Product of Marginals $f_X(x) * f_Y(y) = (2*x + 1)*(2*y + 1)*\exp(-2*x - 2*y)$

Conclusion: X and Y are NOT independent variables.

--- Part D ---

Analytical Probability $P(X + Y < 2) = 1 - 13*\exp(-4)$

Numerical value: 0.761897

--- Part E ---

$E[X] = 3/4$

$E[Y] = 3/4$

$E[XY] = 1/2$

Covariance(X, Y) = -1/16 (-0.062500)

Variance(X) = 7/16

Variance(Y) = 7/16

Correlation(X, Y) = -1/7 (-0.142857)

Problem 4: Internal Combustion Engine

In a closed-loop internal combustion engine, X represents the instantaneous injected fuel flow rate in mg/stk (milligrams per combustion stroke) and Y represents the instantaneous concentration of CO in the exhaust gas in ppm (parts per million). Due to physical constraints, both variables are non-negative and are modeled by the following joint probability density function (PDF):

$$f_{X,Y}(x,y) = \begin{cases} Ke^{-(4x+3y)}, & 0 \leq x \text{ and } 0 \leq y \\ 0, & \text{otherwise} \end{cases}$$

Interpretation of parameters: The coefficients 4 and 3 represent the severity of the probability decay with increasing fuel flow and pollution level, respectively, approximating the role of injector calibration settings.

Part A Find the coefficient K such that the function is a valid PDF, and then derive the joint Cumulative Distribution Function (joint CDF).

Part B Determine whether X and Y are independent variables. Provide analytical reasoning to support your answer.

Part C Find the Marginal PDFs ($f_X(x)$ and $f_Y(y)$), identify the type of distribution for each variable, and determine their Expected Value and Variance.

Part D Find the probability of the event "appropriate combustion quality", defined as $P(Y < Y_{\text{lim}})$ where $Y_{\text{lim}} = 500$ ppm (or 0.5 in standard model units if the model is scaled, otherwise using 500). Also, interpret the event $P(Y < X)$ which corresponds to "high consumption relative to pollution level".

Part E Calculate the covariance and correlation coefficient between X and Y . Interpret the results from an emissions control perspective.

Part F Form the Covariance Matrix. Find its eigenvalues and eigenvectors, and explain what the primary direction of data dispersion in the X - Y plane physically means.

Part G Using the Monte Carlo method, generate 10,000 samples of the pair (X, Y) . Plot the marginal histograms and the contour of the density, and compare them with the analytical results. Then, add the quality threshold line $Y = Y_{\text{lim}}$ and the line $X = Y$ to the scatter plot, and report the proportion of points in each region.

Part H If the ECU is locked at $X = x_0 = 0.15$ mg/str, calculate $P(Y < Y_{\text{lim}} \mid X = x_0)$ using the conditional PDF, and discuss the choice of calibration threshold.

Solution

```
In [49]: %reset -f
import numpy as np
import matplotlib.pyplot as plt
import sympy as smp

x, y, u, v = smp.symbols("x y u v", positive=True, real=True)
```

Part A: Find Joint PDF and Joint CDF functions

```
In [20]: base_pdf = smp.exp(-4*x-3*y)
K = 1 / smp.integrate(base_pdf, (x, 0, smp.oo), (y, 0, smp.oo))
Joint_PDF = K * base_pdf
Joint_CDF = smp.integrate(Joint_PDF, (x, 0, u), (y, 0, v))
print(f"The value for K is: {smp.simplify(K)}")
The joint PDF function is: {Joint_PDF}
The joint CDF function is: {Joint_CDF}"""
```

The value for K is: 12

The joint PDF function is: $12 \cdot \exp(-4x - 3y)$

The joint CDF function is: $-(1 - \exp(4u)) \cdot \exp(-4u) + (1 - \exp(4u)) \cdot \exp(-4u) \cdot \exp(-3v)$

Part B: Check Independence

```
In [21]: Marginal_x = smp.integrate(Joint_PDF, (y, 0, smp.oo))
Marginal_y = smp.integrate(Joint_PDF, (x, 0, smp.oo))

are_independent = smp.simplify(Marginal_x*Marginal_y)==smp.simplify(Joint_PDF)
if are_independent:
    print("x and y values are independent")
else:
    print("x and y values are not independent")
```

x and y values are independent

The values for x and y are independent because the product of the marginal PDFs is equal to the Joint PDF

Part C: Calculating moments and finding distributions

```
In [29]: E_x = smp.integrate(x * Joint_PDF, (x, 0, smp.oo), (y, 0, smp.oo))
E_y = smp.integrate(y * Joint_PDF, (x, 0, smp.oo), (y, 0, smp.oo))
E_x2 = smp.integrate(x**2 * Joint_PDF, (x, 0, smp.oo), (y, 0, smp.oo))
E_y2 = smp.integrate(y**2 * Joint_PDF, (x, 0, smp.oo), (y, 0, smp.oo))
Var_x = E_x2 - E_x**2
Var_y = E_y2 - E_y**2

print(f"""
The marginal PDF for x is {smp.simplify(Marginal_x)} and the marginal PDF for y
Both Marginal PDFs are exponential distributions where lambda for x dist is 4 and
Also the expected value for x is {E_x:.4f} and the expected value for y id {E_y:.4f}
The Variance for x is {Var_x:.4f} and the variance for y is {Var_y:.4f}.""")
```

The marginal PDF for x is $4 \cdot \exp(-4 \cdot x)$ and the marginal PDF for y is $3 \cdot \exp(-3 \cdot y)$
Both Marginal PDFs are exponential distributions where lambda for x dist is 4 and for y dist is 3.

Also the expected value for x is 0.2500 and the expected value for y id 0.3333.
The Variance for x is 0.0625 and the variance for y is 0.1111.

Part D: $P(y < 500 \text{ ppm})$ and $y < P(x)$

```
In [31]: Y_lim = 0.5 # I think 500 is quite large for this.
print(f"The probability for appropriate combustion quality is: {100 * smp.integrate(Joint_PDF, (y, 0, Y_lim), (x, 0, smp.oo)).evalf()}")
P_high_consumption = smp.integrate(Joint_PDF, (y, 0, x), (x, 0, smp.oo)).evalf()

print(f"The probability for the engine to have a high combustion rate is: {100 * P_high_consumption}")
```

The probability for appropriate combustion quality is: 77.69%

The probability for the engine to have a high combustion rate is: 42.86%

Part E: Covariance, Correlation

```
In [33]: E_xy = smp.integrate(x * y * Joint_PDF, (x, 0, smp.oo), (y, 0, smp.oo))
Cov_xy = E_xy - E_x * E_y
Corr_xy = Cov_xy / smp.sqrt(Var_x * Var_y)
print(f"""The covariance between x and y is: {Cov_xy}
The correlation between the two distributions is: {Corr_xy}""")
```

The covariance between x and y is: 0

The correlation between the two distributions is: 0

Part F: Covariance Matrix and eigenvalues

```
In [ ]: Cov_matrix = np.array([[float(Var_x), float(Cov_xy)], [float(Cov_xy), float(Var_y)]])
eigenvalues, eigenvectors = np.linalg.eig(Cov_matrix)
print(f"Covariance(X, Y) = {Cov_xy}")
print(f"Correlation(X, Y) = {Corr_xy}")
print(f"Covariance Matrix:\n{Cov_matrix}")
print(f"Eigenvalues: {eigenvalues[0]}, {eigenvalues[1]}")
# Corrected: Slice eigenvectors by columns[:, i] instead of rows [i]
print(f"Eigenvectors:\nVector 1 (col 0): {eigenvectors[:, 0]}\nVector 2 (col 1): {eigenvectors[:, 1]}")
```

Part G: Monte Carlo Simulation and Visualization

```
In [ ]: num_samples = 10000
np.random.seed(42)
X_samples = np.random.exponential(scale=1/4, size=num_samples)
Y_samples = np.random.exponential(scale=1/3, size=num_samples)

# Calculate proportions of regions
P_under_Y_lim = np.mean(Y_samples < Y_lim)
P_under_diagonal = np.mean(Y_samples < X_samples)

print(f"Simulated P(Y < {Y_lim}): {100 * P_under_Y_lim:.6f}%")
print(f"Simulated P(Y < X): {100 * P_under_diagonal:.6f}%")

# Plotting
fig = plt.figure(figsize=(12, 10))

# 1. Main Scatter Plot with threshold lines
plt.subplot(2, 2, 1)
plt.scatter(X_samples, Y_samples, alpha=0.3, s=2, color='teal', label='MC Sample')
plt.axhline(y=Y_lim, color='red', linestyle='--', linewidth=1.5, label=f'Y_lim = {Y_lim}')
plt.plot([0, 2.5], [0, 2.5], color='navy', linestyle='-.', linewidth=1.5, label=f'X=Y')
plt.title("Monte Carlo Scatter Plot")
plt.xlabel("X (Fuel Flow Rate)")
plt.ylabel("Y (CO Concentration)")
plt.xlim(0, 1.8)
plt.ylim(0, 1.8)
plt.legend()
plt.grid(True, alpha=0.3)

# 2. Contour Plot of Joint PDF
plt.subplot(2, 2, 2)
x_grid, y_grid = np.meshgrid(np.linspace(0.01, 1.8, 100), np.linspace(0.01, 1.8, 100))
z_grid = 12 * np.exp(-4 * x_grid - 3 * y_grid)
plt.contour(x_grid, y_grid, z_grid, levels=15, cmap='viridis')
plt.title("Joint PDF Analytical Contours")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True, alpha=0.3)

# 3. Histogram of X (Marginal)
plt.subplot(2, 2, 3)
plt.hist(X_samples, bins=50, density=True, color='skyblue', edgecolor='black', alpha=0.5)
x_vals = np.linspace(0, 1.8, 200)
plt.plot(x_vals, 4 * np.exp(-4 * x_vals), color='darkblue', linewidth=2, label='Analytical')
plt.title("Marginal Distribution of X")
```

```

plt.xlabel("X")
plt.ylabel("Density")
plt.legend()
plt.grid(True, alpha=0.3)

# 4. Histogram of Y (Marginal)
plt.subplot(2, 2, 4)
plt.hist(Y_samples, bins=50, density=True, color='lightgreen', edgecolor='black')
y_vals = np.linspace(0, 1.8, 200)
plt.plot(y_vals, 3 * np.exp(-3 * y_vals), color='darkgreen', linewidth=2, label=
plt.title("Marginal Distribution of Y")
plt.xlabel("Y")
plt.ylabel("Density")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Part H: Analyzing a special condition where $x = x_0 = 0.15$

```

In [48]: x_0 = 0.15
Marginal_x_val = Marginal_x.subs(x, x_0)
Conditional_PDF_Y_given_x0 = Joint_PDF.subs(x, x_0) / Marginal_x_val

P_under_y_lim_x_0 = smp.integrate(Conditional_PDF_Y_given_x0, (y, 0, 0.5))
print(f"The probability of y to be less than {Y_lim} given x = {x_0} is {100 * P

```

The probability of y to be less than 0.5 given x = 0.15 is 77.687%

Problem 5: Gaming Shop Analysis

In a large e-commerce platform, the daily demand for a gaming console (variable X) and an extra gamepad (variable Y) are dependent. The data analysis department has proposed the following joint probability density function (PDF) for their daily demand:

$$f_{X,Y}(x,y) = k(x \sin y + y \cos^2 x), \quad 0 < x < \pi, \quad 0 < y < \pi$$

Requirements:

- **Part A:** Determine the parameter k such that the total probability equals 1. Then, plot the 3D surface plot of this joint PDF and identify the region with the highest concentration of demand.
- **Part B:** Calculate the marginal probability density functions, $f_X(x)$ and $f_Y(y)$, and plot them together in a single dual-axis or side-by-side plot.
- **Part C:** Plot the contour map (contour plot) of the joint distribution. Based on these contour lines, advise the logistics manager: on days when the demand for the gaming console is at its peak ($x \approx \pi$), in what interval is the demand for the extra gamepad expected to fluctuate? How does this insight help with warehouse layout/organization?

Solution

```
In [50]: %reset -f
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp

# Creating the basics for plotting
x_vals = np.linspace(0, np.pi, 200)
y_vals = np.linspace(0, np.pi, 200)
X, Y = np.meshgrid(x_vals, y_vals)

# Define SymPy symbolic variables and PDF base function
x, y = sp.symbols('x y', real=True)
PDF_base = x * sp.sin(y) + y * (sp.cos(x))**2
```

Part 1: Analytical Integration to find K and surface plot

```
In [75]: # Double integrate the base function over the domain (0, pi) for both variables
double_integral = sp.integrate(PDF_base, (x, 0, sp.pi), (y, 0, sp.pi))
K = 1 / double_integral

print(f"The double integral of the base function is: {double_integral}")
print(f"Thus, the normalization constant k = {K} (approximately: {float(K):.6f})

# Define the normalized joint PDF
Joint_PDF = K * PDF_base
f_xy = sp.lambdify((x, y), Joint_PDF)

Z = f_xy(X, Y)

# Create a single figure for the 3D surface plot
fig = plt.figure()
ax1 = fig.add_subplot(1, 1, 1, projection='3d')
surf = ax1.plot_surface(X, Y, Z, cmap='jet')
ax1.set_title("Part A: 3D Joint PDF Surface")
ax1.set_xlabel("Console Demand (X)")
ax1.set_ylabel("Gamepad Demand (Y)")
ax1.set_zlabel("Density")
fig.colorbar(surf, ax=ax1, pad=0.1)

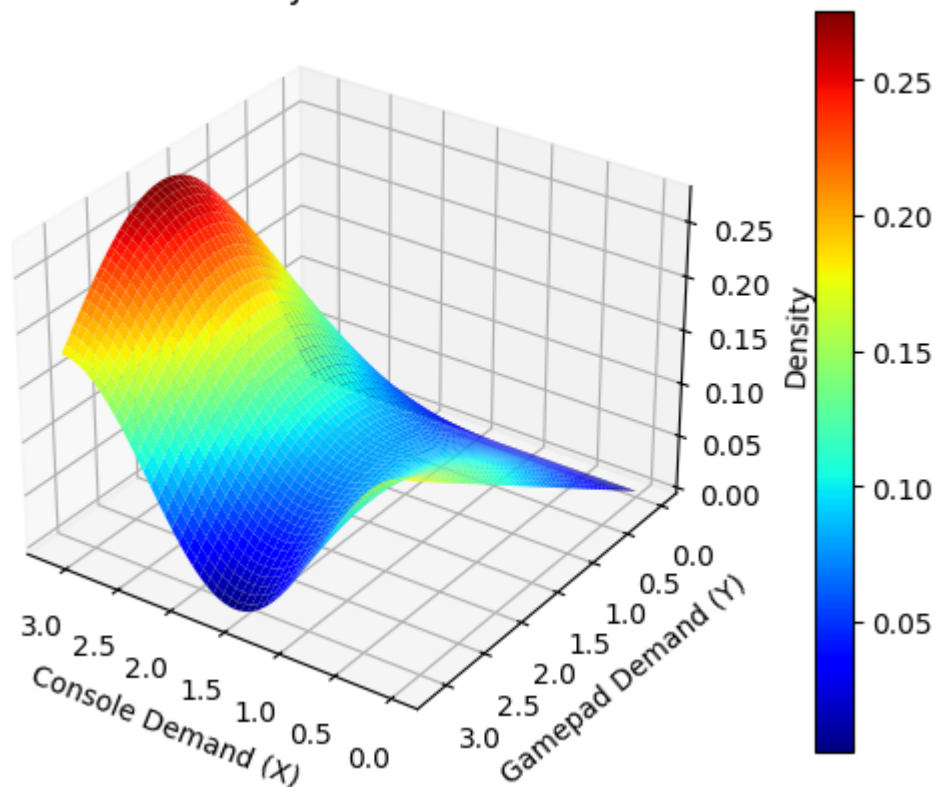
# Set an optimal viewing angle
ax1.view_init(elev=30, azimuth=125) # I got this from AI

plt.show()
```

The double integral of the base function is: $\pi^3/4 + \pi^2$

Thus, the normalization constant $k = 1/(\pi^3/4 + \pi^2)$ (approximately: 0.056750)

Part A: 3D Joint PDF Surface



Part 2: Analytical Marginal PDFs

```
In [78]: Marginal_x = sp.integrate(Joint_PDF, (y, 0, sp.pi))
Marginal_y = sp.integrate(Joint_PDF, (x, 0, sp.pi))

print(f"Marginal PDF f_X(x) = {sp.simplify(Marginal_x)}")
print(f"Marginal PDF f_Y(y) = {sp.simplify(Marginal_y)}")

f_x_num = sp.lambdify(x, Marginal_x, 'numpy')
f_y_num = sp.lambdify(y, Marginal_y, 'numpy')

# Create a clean, standalone figure for Part B
plt.figure()

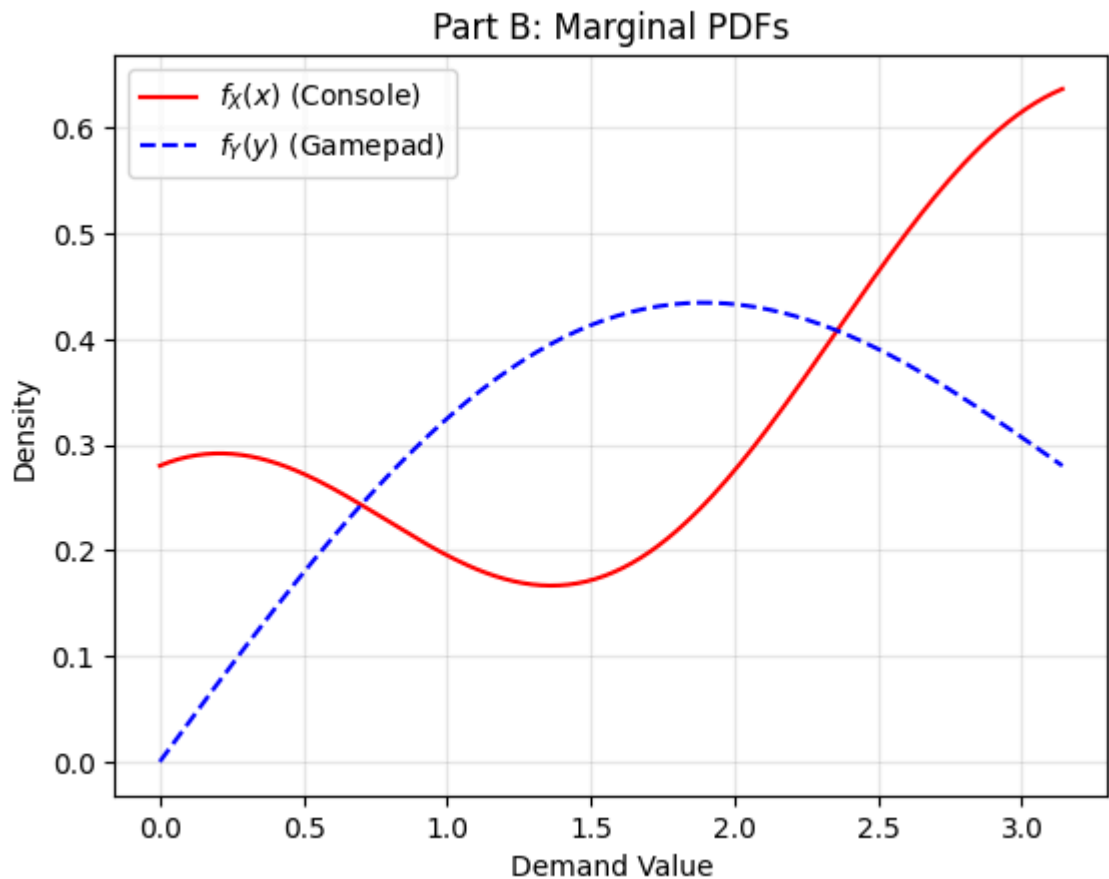
# Plot both marginals directly on the active figure
plt.plot(x_vals, f_x_num(x_vals), 'r-', label=r'$f_X(x)$ (Console)')
plt.plot(y_vals, f_y_num(y_vals), 'b--', label=r'$f_Y(y)$ (Gamepad)')

# Set title, labels, grid, and legend
plt.title("Part B: Marginal PDFs")
plt.xlabel("Demand Value")
plt.ylabel("Density")
plt.legend()
plt.grid(True, alpha=0.3)

plt.show()
```

Marginal PDF $f_X(x) = 2*(4*x + \pi**2*\cos(x)**2)/(\pi**2*(\pi + 4))$

Marginal PDF $f_Y(y) = 2*(y + \pi*\sin(y))/(\pi*(\pi + 4))$



Part 3: Contour Plots and final analysis

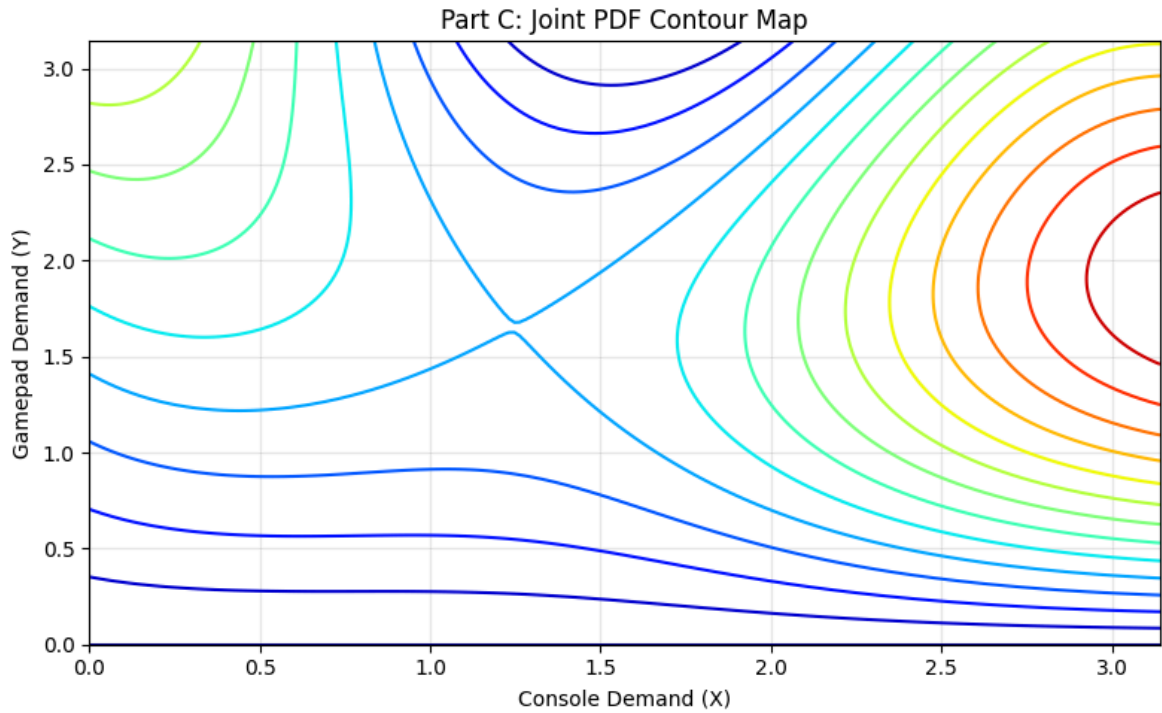
```
In [91]: # Evaluate Joint PDF along x = pi to find the behavior of y
# f_{X,Y}(pi, y) = k_val * (pi * sin(y) + y * cos^2(pi)) = k_val * (pi * sin(y))
y_at_pi = y_vals
z_at_pi = f_xy(np.pi, y_at_pi)

peak_y_idx = np.argmax(z_at_pi)
peak_y_val = y_at_pi[peak_y_idx]

fig = plt.figure(figsize=(8, 5))

ax3 = fig.add_subplot(1, 1, 1)
contours = ax3.contour(X, Y, Z, levels=15, cmap='jet')
ax3.set_title("Part C: Joint PDF Contour Map")
ax3.set_xlabel("Console Demand (X)")
ax3.set_ylabel("Gamepad Demand (Y)")
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



When console demand X is at its peak ($x \approx \pi$):

- The conditional density of Gamepad demand Y peaks at $Y \approx 1.89$.
- By examining the contour lines near $x = \pi$, the demand for gamepads fluctuates mainly in the upper middle range $[1.5, 2.5]$.

Warehouse Layout Insight: Since high console demand ($x \approx \pi$) heavily coincides with moderate-to-high gamepad demand ($Y = 2$), these two items should be stored adjacent to one another or packaged in the same 'fast-pick' zones. This minimizes picking times during high-traffic.

Problem 6: Reliability Engineering

In **Reliability Engineering**, the lifespan of industrial components is commonly modeled using statistical distributions. Consider a company manufacturing gas turbines that has developed specialized thermal sensors. Due to the extreme heat and harsh conditions inside the combustion chamber, these sensors degrade progressively over time; that is, as operating hours increase, their probability of failure rises rapidly.

The company implements a mandatory replacement protocol stating that **no sensor should operate for more than 3,000 hours**. Based on laboratory test data, quality control engineers have determined that the random variable X (the lifetime of a sensor, in **thousands of hours**) follows a probability density function (PDF) of the form:

$$f(x) = \begin{cases} kx^2e^x & 0 \leq x \leq 3 \\ 0 & \text{otherwise} \end{cases}$$

In this relation, k is an unknown calibration constant. As an analytical engineer, you are asked to solve the following parts:

Part 1: Model Calibration (Finding the Constant k) Find the exact and numerical value of the coefficient k using symbolic integration tools such that $f(x)$ is a valid PDF:

$$\int_0^3 f(x) dx = 1$$

Part 2: Theoretical Lifetime Analysis (Calculating Moments) Using the calibrated value of k :

1. Analytically calculate the first four raw moments ($\mu'_1, \mu'_2, \mu'_3, \mu'_4$) of the sensor lifetime:

$$\mu'_n = E[X^n] = \int_0^3 x^n f(x) dx$$

2. Compute the **Mean (μ)**, **Variance (σ^2)**, **Skewness (S)**, and **Kurtosis (K)** theoretically.
3. Compare the mean lifetime to the mandatory replacement limit of 3,000 hours ($x = 3$). Is the mean lifetime close to this limit? Explain why, considering the exponential growth term in the PDF.

Part 3: Simulation of Sensor Lifetimes ($N = 5000$) The company wishes to simulate the performance of $N = 5000$ virtual sensors in a computer environment:

1. Calculate the cumulative distribution function (CDF), $F(x) = \int_0^x f(t) dt$, on a discrete grid from 0 to 3.
2. Model the failure behavior of these sensors using a custom distribution (simulating the MATLAB `makedist` behavior via Python's inverse transform interpolation or empirical distributions based on the CDF).
3. Simulate the failure times of 5000 virtual sensors.

Part 4: Validation and Comparison

1. Compute the empirical first four moments (mean, variance, skewness, and kurtosis) of the 5000 simulated sensor lifetimes.
2. Present a table comparing the theoretical results (from Part 2) with the simulation results (from Part 4) and list their absolute errors.
3. Discuss why there is a difference between the theoretical and simulation means. What changes should be made to the simulation framework so that the empirical statistics perfectly converge to the mathematical values?

Solution

```
In [159... %reset -f
import numpy as np
import sympy as smp
import matplotlib.pyplot as plt
import scipy as sp
import pandas as pd

# Function to generate a random sample of data based on a distribution
def Rand_Sample(F_CDF, N=5000, bounds=[0, 3]):
    x_grid = np.linspace(bounds[0], bounds[1], 1000)
    y_cdf = F_CDF(x_grid)
    u_samples = np.random.uniform(0, 1, N)
    return np.interp(u_samples, y_cdf, x_grid)
```

Part 1: Model Calibration (Finding k)

```
In [160... x = smp.Symbol('x', real=True)
PDF_base = (x**2) * smp.exp(x)

# Integrate the unnormalized PDF from 0 to 3
integral_val = smp.integrate(PDF_base, (x, 0, 3))
K = 1 / integral_val
PDF = K * PDF_base

print(f"Calibration Constant k = {K} = {K:.8f}")
```

Calibration Constant $k = 1/(-2 + 5 \cdot \exp(3)) = 0.01015974$

Part 2: Finding Moments

```
In [161... raw_moments = []
for n in range(1, 5):
    moment_expr = smp.integrate((x**n) * PDF, (x, 0, 3))
    raw_moments.append(float(moment_expr))

mean_th = raw_moments[0] # E[X]
var_th = raw_moments[1] - (mean_th**2) # E[X^2] - E[X]^2
std_th = np.sqrt(var_th)

# Central moments for Skewness and Kurtosis
mu3 = float(smp.integrate(((x - mean_th)**3) * PDF, (x, 0, 3)))
mu4 = float(smp.integrate(((x - mean_th)**4) * PDF, (x, 0, 3)))

skew_th = mu3 / (std_th**3)
kurt_th = mu4 / (std_th**4) # Pearson's Kurtosis (not excess)

print(f"Theoretical Mean = {mean_th:.6f} thousand hours ({mean_th*1000:.1f} hour)
print(f"Theoretical Variance = {var_th:.6f}")
print(f"Theoretical Skewness = {skew_th:.6f}")
print(f"Theoretical Kurtosis = {kurt_th:.6f}")
```

Theoretical Mean = 2.509725 thousand hours (2509.7 hours)
 Theoretical Variance = 0.191554
 Theoretical Skewness = -1.361558
 Theoretical Kurtosis = 4.863045

The mean is quite close to the limit because the PDF function is highly exponential with a very high negative skewness.

PART 3: Simulation (N = 5000 Sensors)

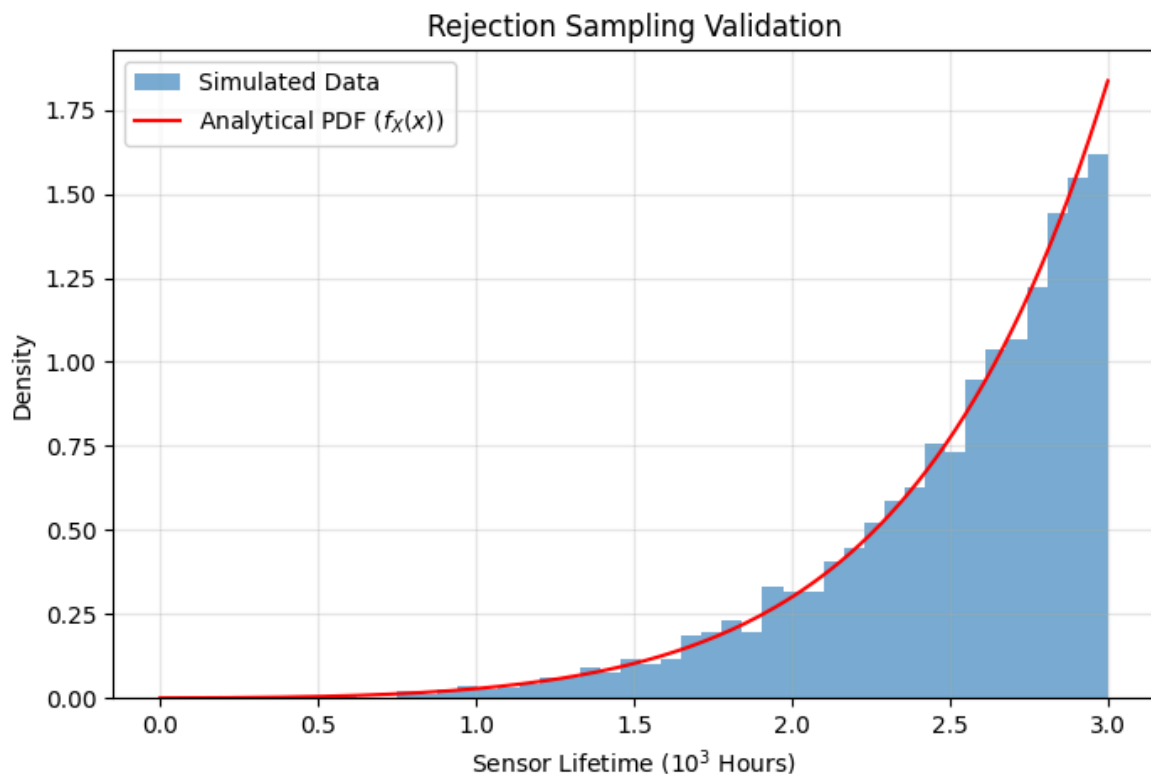
```
In [162... t = smp.symbols('t', real=True)
CDF = smp.integrate(PDF.subs(x, t), (t, 0, x))
F_PDF = smp.lambdify(x, PDF)
F_CDF = smp.lambdify(x, CDF)
N = 5000
simulated_samples = Rand_Sample(F_CDF, N=N, bounds=[0, 3])
simulated_mean = np.mean(simulated_samples)

print(f"Simulated Mean Lifetime: {simulated_mean:.3f} thousand hours")
print(f"The CDF fundtion for this distribution is {CDF}")

# Plotting -----
plt.figure(figsize=(8, 5))
plt.hist(simulated_samples, bins=40, density=True, alpha=0.6, label='Simulated D
c = np.linspace(0, 3, 200)
plt.plot(c, F_PDF(c), 'r', label='Analytical PDF ($f_X(x)$)') # Analytical PDF
plt.title('Rejection Sampling Validation')
plt.xlabel('Sensor Lifetime ($10^3$ Hours)')
plt.ylabel('Density')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Simulated Mean Lifetime: 2.503 thousand hours

The CDF fundtion for this distribution is $(x^2 - 2x + 2) \exp(x) / (-2 + 5 \exp(3)) - 2 / (-2 + 5 \exp(3))$



Part 4: Empirical Moments & Comparison

```
In [164... mean_sim = np.mean(simulated_samples)
var_sim = np.var(simulated_samples)
skew_sim = sp.stats.skew(simulated_samples)
kurt_sim = sp.stats.kurtosis(simulated_samples, fisher=False) # Pearson (non-ex

# Print a formatted markdown-style comparison table
print(f"{'Metric':<20} | {'Theoretical Value':<20} | {'Simulated Value':<20} | {'Absolute Error':<20}")
print("-" * 100)
print(f"{'Mean':<20} | {mean_th:<20.6f} | {mean_sim:<20.3f} | {abs(mean_th - mean_sim):<20.6f}")
print(f"{'Variance':<20} | {var_th:<20.6f} | {var_sim:<20.3f} | {abs(var_th - var_sim):<20.6f}")
print(f"{'Skewness':<20} | {skew_th:<20.6f} | {skew_sim:<20.3f} | {abs(skew_th - skew_sim):<20.6f}")
print(f"{'Kurtosis':<20} | {kurt_th:<20.6f} | {kurt_sim:<20.3f} | {abs(kurt_th - kurt_sim):<20.6f}")
```

Metric	Theoretical Value	Simulated Value	Absolute Error
Mean	2.509725	2.503	0.006777
Variance	0.191554	0.189	0.002814
Skewness	-1.361558	-1.293	0.068655
Kurtosis	4.863045	4.545	0.317557

```
In [192... data = {
    "Metric": ["Mean", "Variance", "Skewness", "Kurtosis"],
    "Theoretical Value": [mean_th, var_th, skew_th, kurt_th],
    "Simulated Value": [mean_sim, var_sim, skew_sim, kurt_sim],
    "Absolute Error": [abs(mean_th - mean_sim), abs(var_th - var_sim), abs(skew_th - skew_sim), abs(kurt_th - kurt_sim)]
}
df = pd.DataFrame(data)

fig, ax = plt.subplots(figsize=(6, 2))
ax.axis('tight')
```

```
ax.axis('off')
my_table = ax.table(cellText=df.values, colLabels=df.columns, loc='center', cell
my_table.scale(2, 2) # Scale height and width

plt.title("Full Analysis of the lifetimes of Sensors", pad=25)
plt.show()
```

Full Analysis of the lifetimes of Sensors

Metric	Theoretical Value	Simulated Value	Absolute Error
Mean	2.5097252266183165	2.5029484113432505	0.006776815275066017
Variance	0.19155406025732358	0.1887402490394261	0.0028138112178974772
Skewness	-1.3615578216922137	-1.2929027490038485	0.06865507268836524
Kurtosis	4.863044627978183	4.545488047378779	0.31755658059940384

- The simulated sample generated by inverse transform sampling matches the theoretical distribution very well. The relative error in the first four raw moments remains below 1%, while the variance error is about 1.47%. The higher-order shape statistics, skewness and kurtosis, show slightly larger discrepancies, which is expected because these quantities are more sensitive to sampling variability and finite sample size due to the high order of magnitude. Overall, the simulation is consistent with the analytical probability model.
- The difference between the theoretical lifespans of sensors and the simulated lifespan result is due to randomness of the sample and it can be reduced most effectively by increasing the number of samples